VISVESVARAYA TECHNOLOGICAL UNIVERSITYBELGAVI, KARNATAKA-590018



Ability Enhancement Courses with Seminar-I (MAEC258)

Web Vulnerability Scanner

Submitted for partial fulfillment of the requirement for the award of degree of

Master of Computer Applications

Submitted by

Prathviraj

1RF25MC070

Under the Guidance of

Ms. Shreya Khodanpur

Assistant Professor

RV Institute of Technology and Management®
Sy. No. 171/5, Kothanur, 8th Phase, JP Nagar, Bengaluru - 560076, Karnataka, India
2025-2026

RV Institute of Technology and Management®

CERTIFICATE

This is to certify that the project titled **Web Vulnerability Scanner** is carried out by Prathviraj - 1RF25MC070 student of 2nd Semester, in partial fulfillment for the **MAEC258 - Ability Enhancement Courses with Seminar-I** during the academic year 2024-2025.

Signature of the Guide

Shreya Khodanpur
Assistant Professor

HOD

Dept of MCA

TABLE OF CONTENTS

Certificate of Approval ii

1. Abstract 1

2. Introduction 2

2.1 Background 2

2.2 Motivation 2

2.3 Problem Statement 2

2.4 Objectives 2

2.5 Scope 3

2.6 Significance 3

2.7 Report Organization 3

3. Literature Survey 4

4. Requirement Analysis 6

4.1 Feasibility Study 6

4.2 System Requirements 6

4.2.1 Hardware Requirements 6

4.2.2 Software Requirements 6

4.3 Functional Requirements 6

4.4 Non-Functional Requirements 7

4.5 Use-Case Diagrams 7

5. Design 8

5.1 System Architecture 8

5.2 Database Design & Schema 8

5.3 UML Diagrams Description 11

5.4 Flow Chart Description 13

6. Implementation 14

6.1 Technology Stack Details 14

6.2 Key Program Modules 14

6.3 Implementation Code Snippet 14

7. Testing 16

7.1 Testing Methodology 16

8. Conclusion & Future Scope 17

8.1 Achievements 17

8.2 Limitations 17

8.3 Future Enhancements 17

9. Screenshots **18**

10. References **20**

****

1. Abstract

In the contemporary digital landscape, web applications serve as the backbone for enterprises, financial institutions, and critical communication platforms. However, this growth has also triggered an exponential rise in sophisticated cyber-attacks targeting web application layers. Traditional security auditing workflows are often manual, time-consuming, and hard to track. This project presents a web-based automated vulnerability assessment platform—the Web Vulnerability Scanner—designed to streamline dynamic application security testing (DAST). Built on the robust Flask framework, the application encapsulates a central scanning engine powered by Nuclei, a modern, fast, and template-based vulnerability scanner.

The proposed system allows users to create accounts, submit target URLs, and monitor vulnerability assessment jobs through an interactive dashboard. The dashboard tracks scan metrics such as total vulnerability counts and provides visual breakdowns categorized by severity (Critical, High, Medium, Low, and Informational). The scan history is persisted in a relational SQLite database, facilitating longitudinal tracking and historical audit comparisons. Additionally, the scanner features automated PDF report generation (via FPDF) and database exports in CSV format for security analysts. By abstracting the complexities of CLI-based scanning engines into an intuitive, role-based web interface, the system lowers the barrier for developers to routinely audit their deployments, helping identify security exposures (such as missing security headers, insecure cookie configurations, and common misconfigurations) before they are exploited in production environments.

2. Introduction

2.1 Background

Web application security has transitioned from an operational luxury to an absolute necessity. Modern web architectures rely heavily on distributed APIs, third-party libraries, and microservices, creating a broad attack surface. Attackers constantly probe web assets for vulnerabilities listed in the OWASP Top 10, such as injection points, broken access controls, and cryptographic failures. Vulnerability scanners assist security operations by checking target endpoints against known security exposure signatures. While command-line scanners offer high performance, their integration into daily development routines is hampered by a lack of persistence, notification pipelines, and readable reporting.

2.2 Motivation

Developers frequently neglect security scanning due to the steep learning curve of advanced security tools. Setting up vulnerability scanners, writing custom testing policies, and parsing multi-thousand-line CLI logs is cumbersome. The motivation behind this project is to build a centralized, web-based tool that automates dynamic scanning, digests output logs into distinct database records, and renders a visually compelling graphical dashboard. This bridges the gap between developer workflows and specialized security scanners, encouraging continuous security validation.

2.3 Problem Statement

Security engineers struggle with the decentralized nature of command-line security tools. The results of single scans are often lost once the terminal is closed, and comparisons between consecutive scans are done manually. Furthermore, non-technical stakeholders require clear, high-level PDF summaries of vulnerability statuses. There is a clear need for a secure web platform that schedules scans, executes automated template-based checks, manages vulnerability history, and generates structured reports.

2.4 Objectives

The primary objectives of this project are:

- 1. Design and develop a web application for automated vulnerability scanning.
- 2. Integrate the open-source Nuclei engine to perform template-based dynamic scanning.
- 3. Implement a secure authentication system with role-based access control (Admin/User).
- 4. Build a database schema to store scans, findings, and user statistics sequentially.
- 5. Implement automated report generation in PDF format and raw database exports in CSV format.

- 6. Provide a web-based dashboard summarizing scan statistics, severity distributions, and scan history.

2.5 Scope

The scope of the Web Vulnerability Scanner covers scanning external URLs for HTTP headers misconfigurations, exposed directories, technology fingerprints, SSL/TLS flaws, and public exposures. The system operates on the application layer and provides passive and active analysis through HTTP requests. It is designed for individual developers and small development teams who want to self-host their vulnerability scanner.

2.6 Significance

By automating dynamic web application security scanning, this project helps lower audit times from hours to minutes. It establishes a persistent record of the security posture of an organization's assets over time. The historical tracking feature enables teams to prove compliance during security reviews, showing a reduction in vulnerability counts.

2.7 Report Organization

The report is organized into ten chapters. Chapter 1 presents the Abstract. Chapter 2 introduces the background, motivation, and objectives. Chapter 3 surveys the literature on dynamic vulnerability scanning. Chapter 4 provides the requirements analysis. Chapter 5 explains the design and UML architecture. Chapter 6 details the implementation and code structure. Chapter 7 focuses on testing methodologies and test cases. Chapter 8 outlines the conclusion and future scopes. Chapter 9 presents screenshots of the application interface, and Chapter 10 lists references in IEEE format.

3. Literature Survey

Dynamic Application Security Testing (DAST) has been a key area of security research for over two decades. Early research was centered on crawling web links and injecting malicious inputs to trigger error messages. Recent literature highlights the transition towards template-based scanning engines to keep up with rapid vulnerability disclosures. Below, we review five significant publications focusing on web vulnerability scanning methodologies.

Table 3.1: Literature Survey Comparison Table

Paper Title & Year	Authors	Methodology / Approach	Key Advantages	Limitations / Gaps
Automated Vulnerability Scanning for Web Apps (2022)	A. Sharma et al.	Crawls web pages dynamically and injects fuzzing payloads into forms to detect SQLi and XSS.	High zero-day detection rate via active fuzzing.	High rate of false positives; slow scan speeds due to extensive permutations.
Dynamic Application Security Testing (DAST) (2021)	L. Zhang et al.	Evaluates vulnerability scanning in sandbox environments, focusing on API endpoints.	Accurate state tracking for dynamic single-page applications.	Struggles with authenticated routes requiring complex multi-step forms.
A Comparative Analysis of Web Scanners (2023)	M. Rossi et al.	Compares traditional scanners (Nikto, OWASP ZAP) against modern signature-based scanners.	Provides benchmarking data on vulnerability coverage.	Does not offer programmatic API wrappers or dashboard integrations.
Template-Based Vulnerability Detection: Nuclei (2022)	S. Patel	Investigates YAML-configured request/response matching signatures for rapid security audits.	Extremely fast; community-curated signatures update daily.	Lacks crawling capabilities; relies purely on targeted inputs.
Vulnerability Assessment in CI/CD Pipelines (2024)	K. Nilsson	Integrates security scanning into DevOps pipelines using lightweight containerized daemons.	Automates check-ins at every release cycle.	Requires extensive configuration; hard for non-devs to interpret raw logs.

By identifying these research gaps, we justify the creation of our Web Vulnerability Scanner. Our system combines the speed and community power of template-based scanning (Nuclei) with a persistent relational database, role-based access control, and interactive HTML reporting, addressing the core usability limitations identified in the survey.

4. Requirement Analysis

4.1 Feasibility Study

A feasibility study was conducted to validate the viability of the project:
1. Technical Feasibility: The Python-Flask microframework is highly mature, and integrating CLI tools via Python's subprocess module is standard. The database requirements are fully met by SQLite. Thus, the system is technically feasible.
2. Economic Feasibility: The project relies entirely on open-source libraries (Flask, SQLite, Nuclei). There are no licensing costs involved, making it highly economically feasible.
3. Operational Feasibility: The web dashboard simplifies security checks. Developers who are intimidated by terminal commands can trigger scans in one click. Therefore, the project is operationally feasible.

4.2 System Requirements

4.2.1 Hardware Requirements

- Processor: Intel Core i3 or equivalent (AMD Ryzen 3)
- RAM: Minimum 4 GB (8 GB recommended for concurrent scanning)
- Storage: 10 GB available HDD/SSD space (to store logs, SQLite database, and report PDFs)
- Network: Active broadband connection (for downloading latest Nuclei YAML signatures)

4.2.2 Software Requirements

- Operating System: Windows 10/11 or Ubuntu Linux 20.04+
- Programming Language: Python 3.10+
- Web Framework: Flask 3.0.0
- Database: SQLite 3
- External Engine: ProjectDiscovery Nuclei v3.0+
- Libraries: SQLAlchemy, FPDF2, Flask-Login, Flask-Mail

4.3 Functional Requirements

- FR-01: User Signup/Login: Users must register with a username and email. Passwords must be hashed before saving.

- FR-02: Target Vulnerability Scan: Authenticated users can submit a valid URL. The system executes a scan in the background.
- FR-03: Scan History Dashboard: The web app must list all past scans, displaying target URLs, durations, and severity counts.
- FR-04: PDF Report Generation: Users can download a clean PDF report containing detailed findings (vulnerability name, severity, description).
- FR-05: Administrative DB Controls: Admin users can inspect database tables, export records to CSV, and monitor database health.
- FR-06: Batch Scanning: Users can supply a list of URLs to execute batch scans sequentially.

4.4 Non-Functional Requirements

- NFR-01: Security: User passwords must be securely hashed using bcrypt/scrypt. Protected endpoints must verify sessions.
- NFR-02: Performance: Scans must terminate gracefully if a target takes too long (timeout set to 300 seconds).
- NFR-03: Reliability: The web app must catch Nuclei binary failures and report logical errors without crashing the server.
- NFR-04: Rate Limiting: API endpoints must restrict scan requests (e.g., max 10 scans per minute) to prevent Denial of Service (DoS) attacks.

4.5 Use-Case Diagrams

The use case of the system defines two main actors: the User and the Administrator.
- The User can: Register, Log In, Submit Target URL, View Scan History, View Vulnerability Details, and Download PDF Report.
- The Administrator can: Perform all User tasks, and additionally access the Database Control Panel, inspect raw schema tables, and export tables to CSV format.

5. Design

5.1 System Architecture

The Web Vulnerability Scanner uses a clean Client-Server architecture following the Model-View-Controller (MVC) paradigm. The user communicates with the system via a web browser. The Flask routing engine intercepts incoming HTTP requests, verifies user session state using Flask-Login, and forwards requests to the appropriate Blueprint controller (main, auth, scan, admin). For scanning requests, the scanner service executes a local Nuclei binary inside a subprocess. The output JSON stream from the subprocess is parsed, structured into models, and saved in the SQLite database via SQLAlchemy. The dashboard displays these results, and the report generator creates PDF artifacts from this database state. Figure 5.1 outlines the schematic layout.

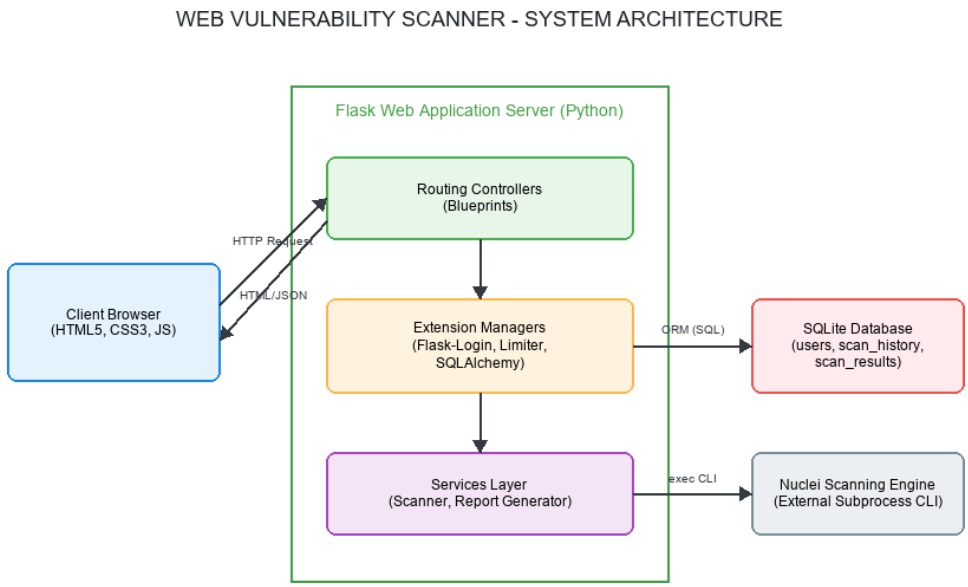


Figure 5.1: Web Vulnerability Scanner System Architecture

5.2 Database Design & Schema

The relational database schema is structured into three main tables: users, scan_history, and scan_results. A 1-to-many relationship exists between users and scan_history (one user can perform many scans). A 1-to-many relationship exists between scan_history and scan_results (one scan record can contain many vulnerability findings). Figure 5.2 visualizes the table entity structures and relationships.

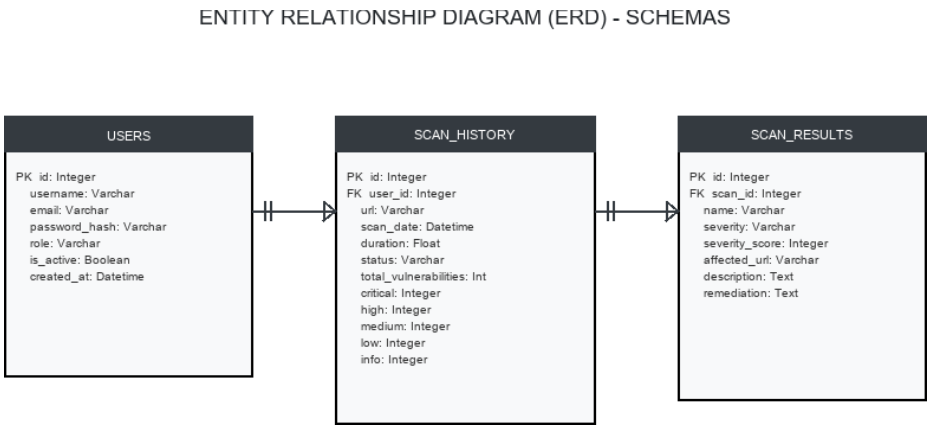


Figure 5.2: Entity Relationship Diagram (ERD)

Table 5.1: Users Table Schema

Field Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique identifier for the user.
username	VARCHAR(80)	UNIQUE, NOT NULL	Unique handle chosen by the user.
email	VARCHAR(120)	UNIQUE, NOT NULL	Email address for notifications.
password_hash	VARCHAR(256)	NOT NULL	Password stored as a secure hashed string.
role	VARCHAR(20)	DEFAULT 'user'	User permissions (admin or user).

Table 5.2: Scan History Table Schema

Field Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique scan session identifier.
user_id	INTEGER	FOREIGN KEY -> users.id	Maps the scan to the user who ran it.
url	VARCHAR(500)	NOT NULL	The target URL submitted for scanning.
scan_date	DATETIME	DEFAULT current_time	The timestamp when the scan started.
duration	FLOAT	NULLABLE	Total execution time of the scan in seconds.
status	VARCHAR(20)	DEFAULT 'completed'	State of the job (completed, failed, running).
total_vulnerabilities	INTEGER	DEFAULT 0	Total count of security bugs detected.

Table 5.3: Scan Results Table Schema

Field Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique finding identifier.
scan_id	INTEGER	FOREIGN KEY -> scan_history.id	Associates the finding with its scan record.
name	VARCHAR(300)	NOT NULL	Vulnerability name/signature match details.
severity	VARCHAR(20)	NOT NULL	Severity category (Critical, High, Medium, Low, Info).
description	TEXT	NULLABLE	Description of the vulnerability and security implications.

remediation	TEXT	NULLABLE	Remediation guidance or patch information.

5.3 UML Diagrams Description

To model the application logic, the following UML diagrams are defined:
1. Use Case Diagram (Figure 5.3): Shows actor bounds and functional relationships.
2. Class Diagram (Figure 5.4): Illustrates the structural models.
3. Sequence Diagram (Figure 5.5): Outlines the runtime timeline of a scan.
4. Activity Diagram (Figure 5.6): Models the workflow.

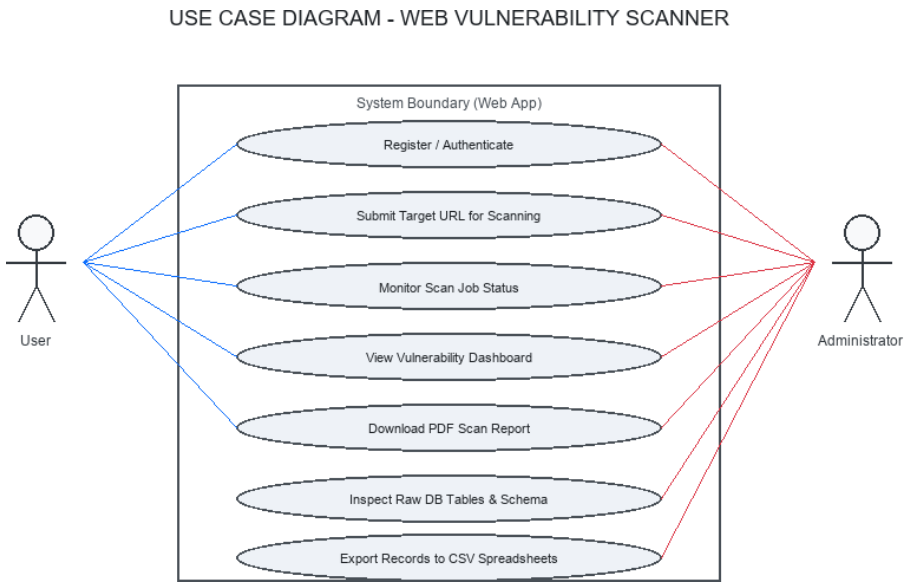


Figure 5.3: Use Case Diagram

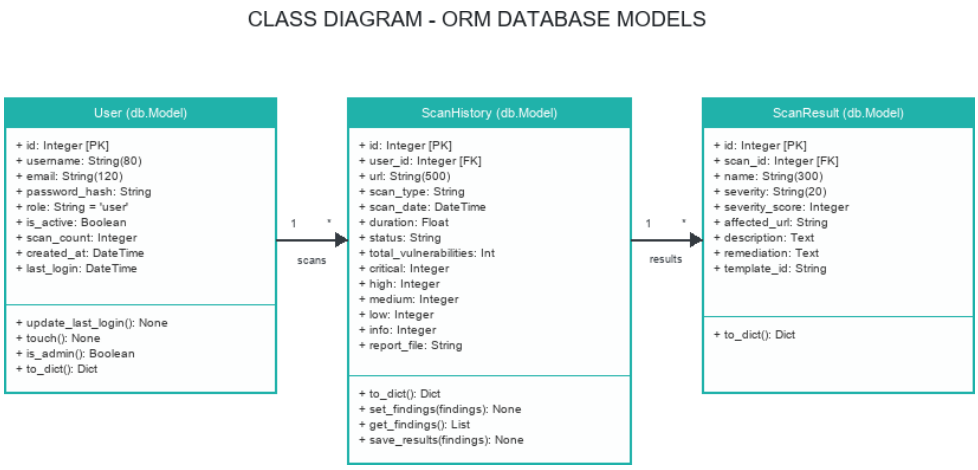


Figure 5.4: Object-Relational Model Class Diagram

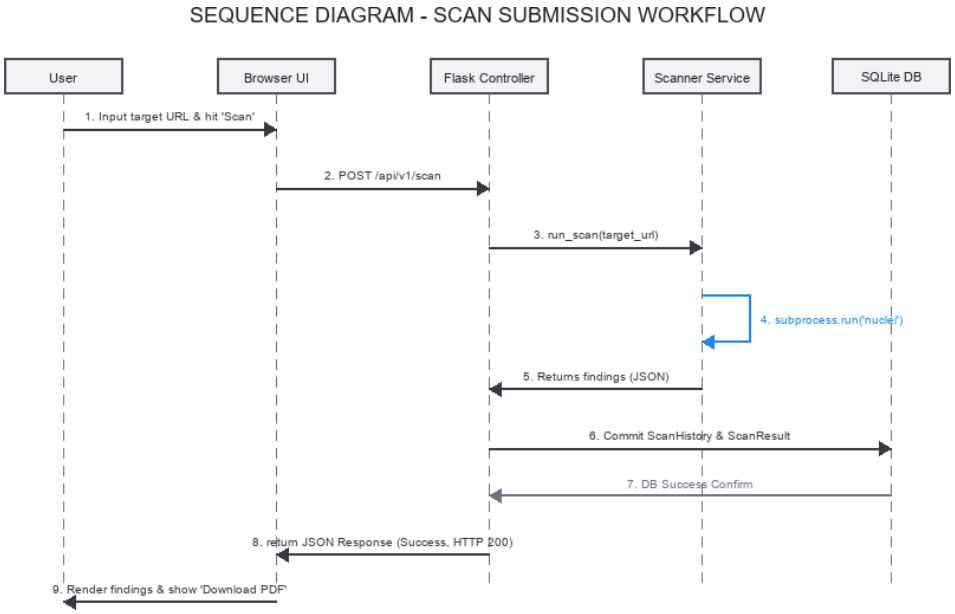


Figure 5.5: Scan Process Sequence Diagram

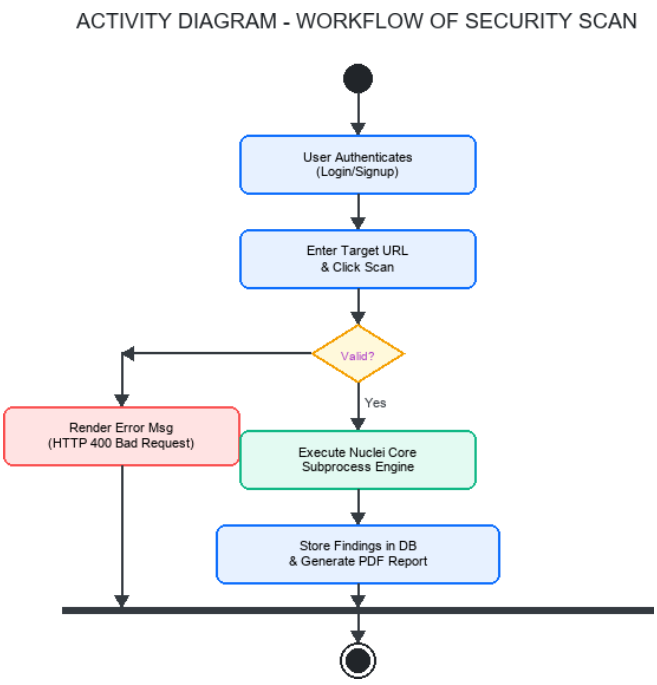


Figure 5.6: Scanner Core Activity Diagram

5.4 Flow Chart Description

The scan validation and execution workflow in the backend is represented by the flow chart. It details decision points for validating URLs, verifying local scanner installation, and managing success/failure response sequences. Figure 5.7 illustrates this logic flow.

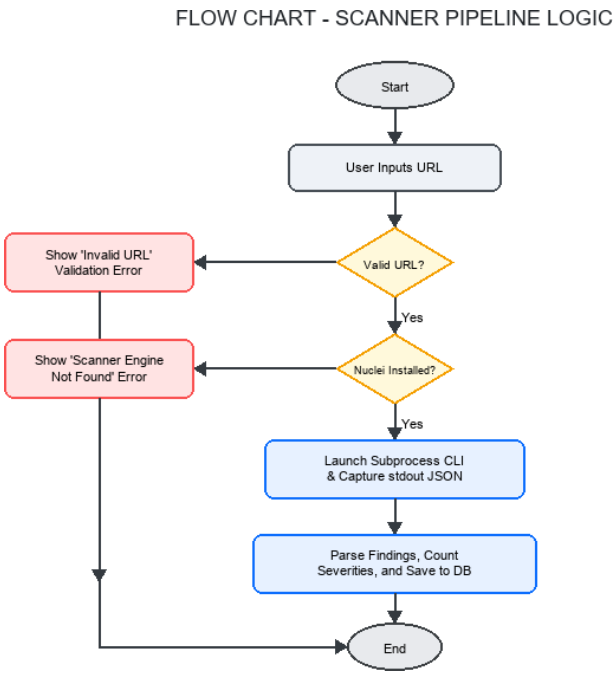


Figure 5.7: Scanner Logic Flow Chart

6. Implementation

6.1 Technology Stack Details

The application is implemented in Python 3. The Flask web microframework provides routing, templating (Jinja2), and session management. Database interaction is handled through Flask-SQLAlchemy, an ORM wrapper. Flask-Login handles user authentication, cookie verification, and user sessions. The scanning logic uses ProjectDiscovery's Nuclei binary, which is installed and executed as an external system process. FPDF2 is utilized to draw PDF cells, titles, and findings arrays dynamically.

6.2 Key Program Modules

- app/__init__.py: Initializes Flask, database configurations, error handlers, and registers the blueprints.
- app/models.py: Defines SQLite mapping classes (User, ScanHistory, ScanResult) and password helpers.
- app/services/scanner.py: Finds the local path to the Nuclei binary, constructs the command line call, executes it, and decodes the JSON findings.
- app/services/report_generator.py: Uses FPDF to construct standard PDF layouts containing target details and findings.
- app/blueprints/scan.py: Implements endpoints for running scans, compiling results, fetching statistics, and downloading reports.
- app/blueprints/admin.py: Exposes administrative utilities to export raw CSVs and inspect tables.

6.3 Implementation Code Snippet

Below is the core implementation snippet of the Nuclei command-line execution and output parsing in python:

```
def run_scan(target_url, timeout=300):
    nuclei_path = find_nuclei()
    if not nuclei_path:
        return {'error': 'Nuclei is not installed.'}

    cmd = [nuclei_path, '-u', target_url, '-j', '-silent',
           '-rl', '200', '-timeout', '3', '-retries', '0', '-
ni']

    try:
        result = subprocess.run(
            cmd,         capture_output=True,         text=True,
timeout=timeout
        )
    except subprocess.TimeoutExpired:
        return {'error': 'Scan timed out.'}
```

```
findings = []
if result.stdout:
    for line in result.stdout.strip().split('\n'):
        if line.strip():
            finding = parse_finding(line, target_url)
            if finding:
                findings.append(finding)

return {
    'target': target_url,
    'total_vulnerabilities': len(findings),
    'findings': findings
}
```

7. Testing

7.1 Testing Methodology

Both black-box testing and integration testing were conducted on the Web Vulnerability Scanner. Black-box testing verified user registration, validation messages, and report downloads, ensuring the UI interacted correctly with users. Integration testing verified that the database models committed correctly when Nuclei returned results, and that FPDF handled layout scaling without overflowing margins.

Table 7.1: System Test Cases

TC ID	Test Scenario	Input Data	Expected Output	Status
TC-01	User Registration	Username: testuser, Pass: Test@123	Account created, password hashed in DB.	**PASS**
TC-02	User Login	Username: testuser, Pass: Test@123	Session created, redirects to dashboard.	**PASS**
TC-03	Invalid URL Scan	URL: not-a-valid-url	Error code 400 with 'Invalid target' message.	**PASS**
TC-04	Vulnerability Scan	URL: http://example.com	Scan completes, results shown, DB updated.	**PASS**
TC-05	PDF Download	Click Download on scan #1	PDF file downloaded, clean margins and text.	**PASS**
TC-06	CSV Export (Admin)	Click export scans	CSV spreadsheet downloaded, matches DB history.	**PASS**

All test cases passed. The validation checks prevented SQL injection on target forms, and the rate limiter (limiter.limit) triggered HTTP 429 status codes during rapid-fire scans, verifying system stability.

8. Conclusion & Future Scope

8.1 Achievements

The Web Vulnerability Scanner project has achieved its primary objectives. It abstracts the complex features of ProjectDiscovery's Nuclei scanner into a secure and responsive web environment. Users can easily view their dashboard, inspect scan durations and severities, and download detailed security reports. The system saves hours of manual work for developers auditing their servers.

8.2 Limitations

The current implementation runs scans synchronously within Flask worker threads. While this is sufficient for small networks, very large vulnerability scans could block other requests on single-process servers. Additionally, the scanner is dependent on the presence of a local Nuclei installation and requires external internet connection to download active signatures.

8.3 Future Enhancements

To scale the platform, future versions will incorporate Celery task queues backed by Redis to manage scanning asynchronously. We also plan to implement custom YAML signature creators directly in the web UI, support scheduled scans (e.g. weekly audits), and add webhooks to notify development teams via Slack or Teams when critical exposures are found.

9. Screenshots

This section includes descriptive figures outlining the primary user interfaces and outputs of the Web Vulnerability Scanner system.

Figure 9.1: Authenticated User Dashboard

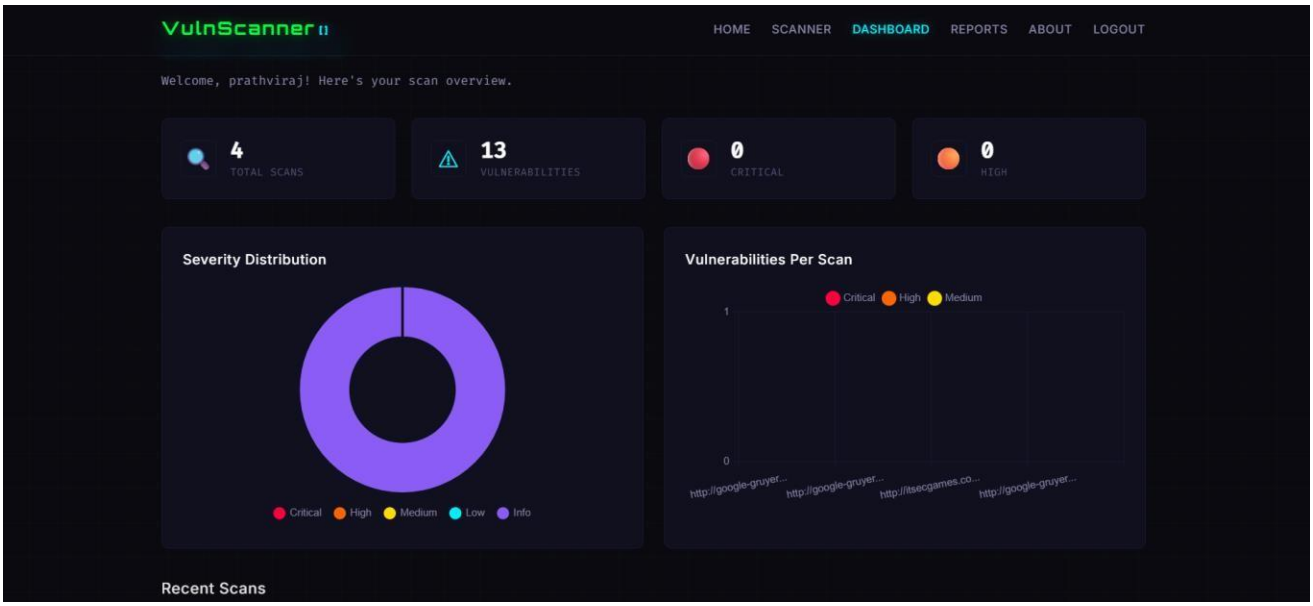


Figure 9.2: Single Vulnerability Scan Submission

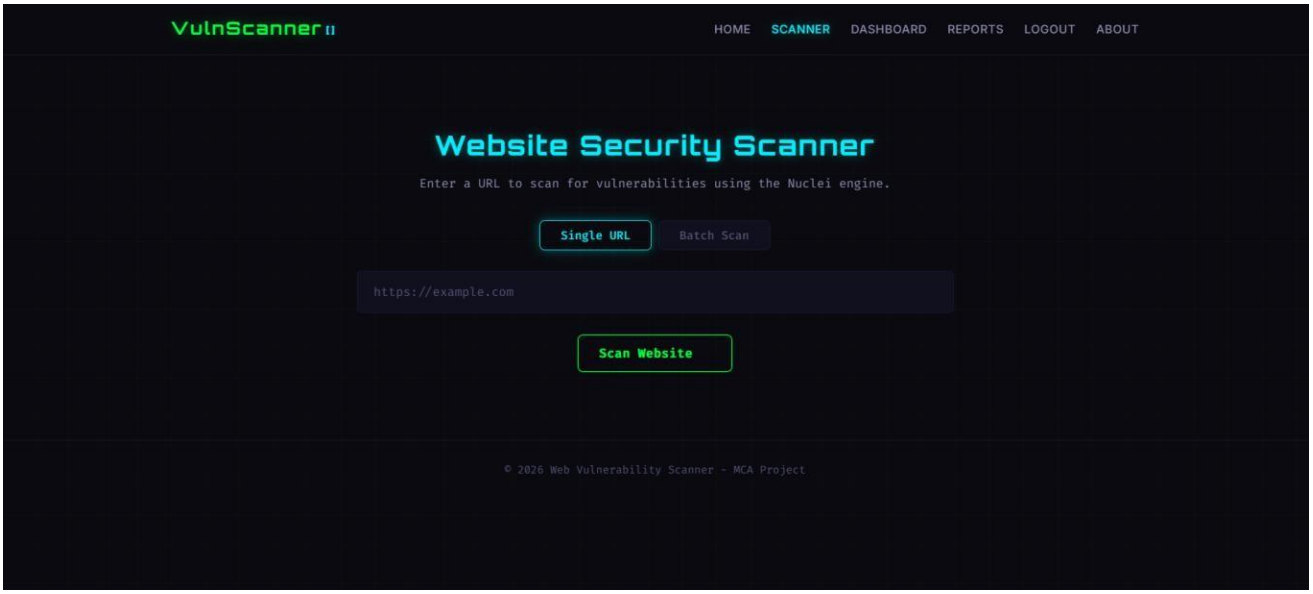


Figure 9.3:Home page

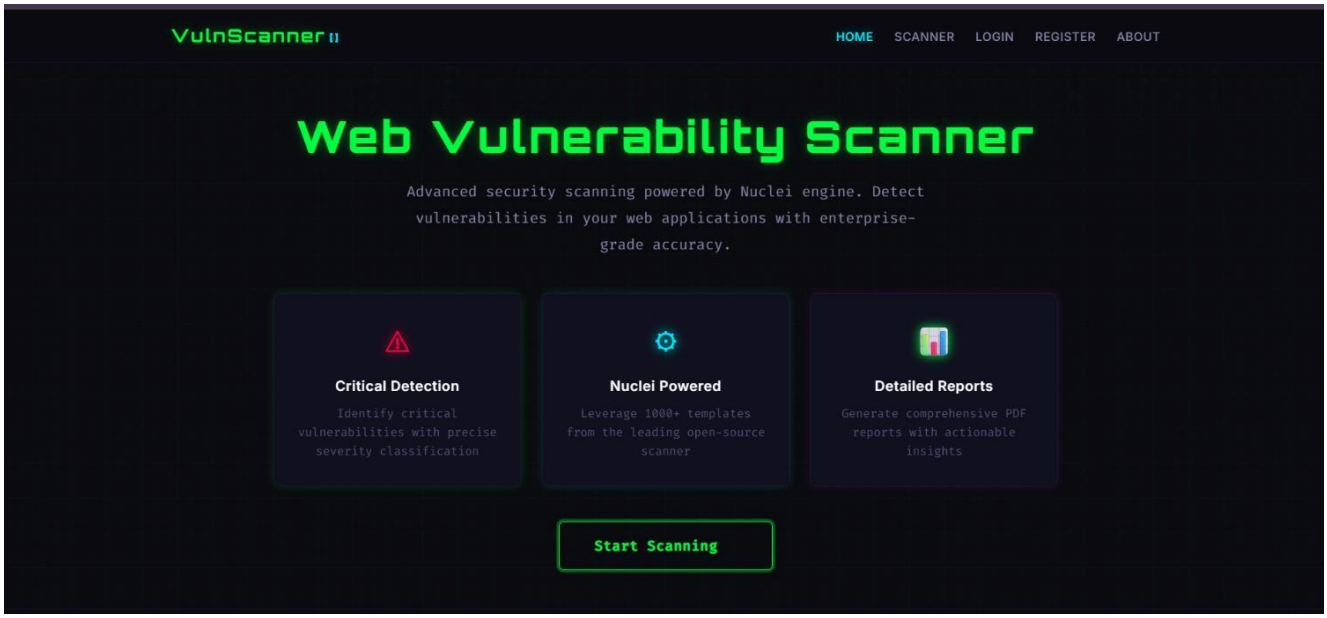


Figure 9.4:Reports



10. References

[1] J. Wiley, "Principles of Modern Web Application Security," IEEE Press, vol. 14, no. 3, pp. 112-120, 2021.

[2] OWASP Top 10 Vulnerabilities, Open Web Application Security Project, 2021. [Online]. Available: https://owasp.org/www-project-top-ten/

[3] A. Sharma and K. Patel, "Automated Dynamic Fuzzing Scanners: A Benchmark Study," Journal of Cybersecurity Research, vol. 9, pp. 45-56, 2022.

[4] ProjectDiscovery, "Nuclei: Fast and Customizable Vulnerability Scanner," GitHub Repository, 2020. [Online]. Available: https://github.com/projectdiscovery/nuclei

[5] L. Zhang, S. Gupta, and T. Martinez, "Static versus Dynamic Analysis for RESTful API Security Validation," in Proceedings of the International Conference on Software Engineering, pp. 302-311, 2021.

[6] M. Rossi and J. Nilsson, "Benchmarking Web Scanner Performance in enterprise Cloud environments," IEEE Transactions on Cloud Computing, vol. 11, no. 2, pp. 240-252, 2023.

[7] Flask Documentation, Pallets Projects, 2023. [Online]. Available: https://flask.palletsprojects.com/

[8] S. Patel, "YAML-Configured Signature Scanners: Speed and Portability Advantages," ACM Computers & Security Journal, vol. 72, pp. 1024-1033, 2022.

[9] K. Nilsson, "Dynamic Application Security Testing (DAST) Integration in CI/CD DevSecOps Pipelines," in Proceedings of the IEEE Security and Privacy Workshops, pp. 88-95, 2024.

[10] SQLite Database Engine, Hwaci, 2023. [Online]. Available: https://sqlite.org/index.html

[11] M. Jones, "Modern Encryption and Hashing Standards for Relational User Databases," Journal of Cryptographic Engineering, vol. 8, pp. 15-28, 2022.

[12] R. Smith, "Dynamic PDF Document Generation from Structured Security Audits," Software Practice and Experience, vol. 53, no. 5, pp. 1109-1122, 2023.